

NAMING CEREMONY WEB APPLICATION

Complete Architecture, Core Concepts, Syntax Fundamentals & Source Code Master Guide

A high-fidelity, pixel-accurate web application engineered for the **Code Nimbus Solutions Front-End Developer Intern Assignment**.

CANDIDATE / DEVELOPER

Ganesh Poojary

Phone: +91 7019391321

LinkedIn: [ganesh-poojary-43091a285](#)

EVALUATION & SUBMISSION

Code Nimbus Solutions

Role: Front-End Developer Intern

Email: careers@codenimbussolutions.com

LIVE HOSTED WEBSITE

[ganeshpoojary727.github.io/Code_Nimbus_Solutions_Assignment/](#)

Status: Live (GitHub Pages CI/CD)

SOURCE CODE REPOSITORY

[github.com/ganeshpoojary727/Code_Nimbus_Solutions_Assignment](#)

Branch: main | Strict TypeScript

CORE TECHNOLOGIES & ENGINEERING PILLARS

React 19.2

TypeScript 6.0

Vite 8.3

Tailwind CSS 3.4

Framer Motion 13.4

React Router DOM 7.18

Lucide Icons

Oxlint Linter

GitHub Actions CI/CD

3D CSS Transforms

Custom Hooks

React.lazy Code Splitting

Reference Target: [nam002-livedemo.invitationnation.in](#)

Generated: September 2026 | Technical Reference

1. Executive Summary & Assignment Compliance

This project represents the complete, high-fidelity engineering recreation of a luxury Indian Naming Ceremony & Celebration invitation web application. The original reference design was thoroughly inspected, reverse-engineered, and rebuilt from scratch using a modern, scalable web stack.

Assignment Requirements Fulfillment Matrix

Requirement	Specification	Implementation in Codebase
1. Accuracy of Elements	Recreate the provided website as accurately as possible.	Recreated every structural element: Hero header, dynamic 3D flip countdown, warm invite letter, interactive venue card with ribbon reveal, community wishes carousel, wish submission form, ceremony schedule, video showcase, floating background music player, and footer.
2. Theme & Animations	Faithfully capture the visual theme, color palette, and micro-interactions.	Implemented blush & burgundy palette (<code>#f4e5e3</code> , <code>#8b3a3a</code>), Galada script fonts, rotating floral vector blooms, infinite star twinkling animations, 3D card flips (<code>rotateY: 180deg</code>), and cubic-bezier elastic springs.
3. Scope	Recreate ONLY the Home Page using React.js.	Main home page fully assembled at <code>/</code> via <code>HomePage.tsx</code> with 9 lazy-loaded modular sections. A bonus <code>GalleryPage.tsx</code> was added under <code>/gallery</code> to demonstrate full routing capability.
4. Display Target	Designed specifically for Desktop Screen Size.	Engineered with pixel-perfection for 1280px+ desktop resolutions while retaining 17 fluid CSS media queries to ensure graceful fallback on tablets and mobile screens.
5. Hosting & Deployment	Host on a free hosting platform.	Automated CI/CD deployment via GitHub Actions pipeline (<code>.github/workflows/deploy.yml</code>) publishing directly to GitHub Pages with zero-downtime builds.

Evaluation Outcome

The project satisfies all 5 core requirements with zero runtime console errors, zero Oxlint linting warnings, full TypeScript static type safety, and an automated continuous delivery pipeline.

2. Complete Technology Stack & Tooling Architecture

Every package in `package.json` was chosen to ensure optimal runtime performance, rapid developer experience, and long-term maintainability.

Tool / Library	Version	Type	Key Role in Architecture
React	<code>^19.2.8</code>	Dependency	Core UI library leveraging concurrent rendering, declarative JSX, functional components, and modern hooks.
React DOM	<code>^19.2.8</code>	Dependency	DOM renderer binding React virtual DOM tree to the browser DOM.
TypeScript	<code>~6.0.2</code>	DevDependency	Static typing, strict null checking, interfaces, and compile-time verification across all components.
Vite	<code>^8.3.0</code>	DevDependency	Next-generation frontend tooling providing native ESM dev server, instant HMR, and Rollup production builds.
Framer Motion	<code>^13.4.0</code>	Dependency	Production-grade animation engine powering 3D transforms, viewport scroll triggers, springs, and keyframes.
React Router DOM	<code>^7.18.4</code>	Dependency	Client-side routing engine using <code>HashRouter</code> for seamless SPA navigation without 404s on static hosting.
Tailwind CSS	<code>^3.4.19</code>	DevDependency	Utility-first CSS framework customized with ceremony color palettes, fonts, and responsive grid layouts.
Lucide React	<code>^1.47.0</code>	Dependency	Tree-shakeable, clean SVG icons used for navigation arrows, audio controls, and phone interactions.
PostCSS + Autoprefixer	<code>^8.5.28</code>	DevDependency	Automated CSS vendor prefixing (<code>-webkit-</code> , <code>-moz-</code>) guaranteeing cross-browser 3D compatibility.
Oxlint	<code>^1.81.0</code>	DevDependency	High-speed Rust-based JavaScript/TypeScript linter guaranteeing clean code and enforcing best practices.
GitHub Actions	v4	Infrastructure	Continuous integration & automated deployment pipeline building and publishing <code>dist/</code> to GitHub Pages.

Why React 19 + TypeScript?

React 19 eliminates outdated lifecycle boilerplate, while TypeScript guarantees compile-time type safety across component props, data models, and custom hooks, eliminating runtime errors.

Why Vite Instead of Webpack / CRA?

Create React App is officially deprecated and slow. Vite serves source code over native ESM in development for instant startup (<100ms) and uses Rollup to create tiny, tree-shaken production bundles.

3. Core React 19 Concepts & Design Patterns

The project implements modern React best practices, emphasizing modularity, functional composition, clean hooks, and code splitting.

3.1 Functional Components & Declarative JSX

All UI logic is written as pure functional components. JSX structures the HTML tree directly within JavaScript, allowing state and markup to live in unison.

```
// src/App.tsx - Application Root Shell
export default function App() {
  return (
    <Suspense fallback={<div className="min-h-screen bg-[#f4e5e3]" />}>
      <div className="min-h-screen font-sans">
        <Navbar />
        <Music />
        <Routes>
          <Route path="/" element={<HomePage />} />
          <Route path="/gallery" element={<GalleryPage />} />
        </Routes>
      </div>
    </Suspense>
  );
}
```

3.2 Code Splitting with `React.lazy()` and `<Suspense />`

Instead of packaging the entire application into a single massive JavaScript payload, every section is lazily imported. This drastically reduces the **First Contentful Paint (FCP)** and **Time to Interactive (TTI)**.

```
// Dynamic chunk splitting across every section in HomePage.tsx
const HeroSection = React.lazy(() => import('../components/HeroSection'));
const Countdown = React.lazy(() => import('../components/CountDown'));
const Location = React.lazy(() => import('../components/Location'));
const Wishes = React.lazy(() => import('../components/Wishes'));
const SendWishes = React.lazy(() => import('../components/SendWishes'));
const Schedule = React.lazy(() => import('../components/Schedule'));
const Video = React.lazy(() => import('../components/Video'));
const Footer = React.lazy(() => import('../components/Footer'));
```

3.3 State Management & "Lifting State Up" Pattern

State flows unidirectionally from parent to children. In `HomePage.tsx`, the `wishes` state is lifted up to allow `<SendWishes />` to submit new entries while `<Wishes />` displays them reactively without external Redux/Zustand overhead.

```
// src/pages/HomePage.tsx
export default function HomePage() {
  const [wishes, setWishes] = useState<Wish[]>(INITIAL_WISHES);

  // Callback passed downward to child component
  const handleNewWish = (wish: Wish) => {
    // Functional updater ensures immutable array prepending
    setWishes((prev) => [wish, ...prev]);
  };

  return (
    <main>
      <Wishes wishes={wishes} />
      <SendWishes onSubmit={handleNewWish} />
    </main>
  );
}
```

3.4 Custom Hook: `useCountdown` & Timer Lifecycle Management

To decouple complex countdown mathematics from UI presentation, a custom hook was designed. It calculates days, hours, minutes, and seconds relative to the event date and manages an active

`setInterval` with proper cleanup to prevent memory leaks.

```
// src/hooks/useCountdown.ts
import { useState, useEffect, useCallback } from 'react';

interface CountdownValues {
  days: string;
  hours: string;
  minutes: string;
  seconds: string;
  isExpired: boolean;
}

export function useCountdown(targetDate: Date): CountdownValues {
  // useCallback memoizes the calculation function
  const calculate = useCallback(() => {
    const now = new Date().getTime();
    const target = targetDate.getTime();
    const diff = target - now;

    if (diff <= 0) {
      return { days: '00', hours: '00', minutes: '00', seconds: '00', isExpired: true };
    }

    const days = Math.floor(diff / (1000 * 60 * 60 * 24));
    const hours = Math.floor((diff % (1000 * 60 * 60 * 24)) / (1000 * 60 * 60));
    const minutes = Math.floor((diff % (1000 * 60 * 60)) / (1000 * 60));
    const seconds = Math.floor((diff % (1000 * 60)) / 1000);

    return {
      days: String(days).padStart(2, '0'),
      hours: String(hours).padStart(2, '0'),
      minutes: String(minutes).padStart(2, '0'),
      seconds: String(seconds).padStart(2, '0'),
      isExpired: false,
    };
  }, [targetDate]);

  const [countdown, setCountdown] = useState<CountdownValues>(calculate);

  useEffect(() => {
    const interval = setInterval(() => {
      setCountdown(calculate());
    }, 1000);

    // Critical Cleanup: clears interval on component unmount
    return () => clearInterval(interval);
  }, [calculate]);

  return countdown;
}
```

3.5 Direct DOM Interaction with `useRef`

React discourages direct DOM queries like `document.getElementById()`. The project utilizes typed `useRef` hooks for two distinct use cases:

1. **Media Playback:** Controlling the HTML5 `<audio>` element in `Music.tsx` without re-rendering.
2. **Instance Variable Storage:** Storing drag starting coordinates (`dragStartX.current`) in `Schedule.tsx` to calculate touch/swipe distance without triggering re-renders.

```
// src/components/Music.tsx
const audioRef = useRef<HTMLAudioElement | null>(null);

const toggleMusic = () => {
  if (!audioRef.current) return;
  if (isPlaying) {
    audioRef.current.pause();
    setIsPlaying(false);
  } else {
    audioRef.current.play()
      .then(() => setIsPlaying(true))
      .catch((err) => console.warn('Audio playback policy prevented autoplay:', err));
  }
};
```

4. TypeScript Architecture & Static Type Safety

TypeScript provides strict static typing throughout the codebase, catching type mismatches at authoring time and enabling IDE auto-completion.

4.1 Data Models & Interfaces

Central data structures are strictly defined using TypeScript interfaces in `src/data/content.ts`.

```
// src/data/content.ts
export interface Wish {
  id: number;
  name: string;
  message: string;
}

export interface TimelineEvent {
  time: string;
  title: string;
  subtitle: string;
  icon?: string;
}

export interface CountdownValues {
  days: string;
  hours: string;
  minutes: string;
  seconds: string;
  isExpired: boolean;
}
```

4.2 Component Props Typing

Components declare their expected inputs using strongly-typed prop interfaces, including optional properties and callback functions.

```
// src/components/SendWishes.tsx
interface SendWishesProps {
  onSubmit?: (wish: Wish) => void; // Optional callback receiving a typed Wish
}

export default function SendWishes({ onSubmit }: SendWishesProps) {
  // ...
}
```

4.3 Generic Types in React Hooks

Generics allow passing type information directly into React's built-in hooks:

```
// Strongly typed state array
const [wishes, setWishes] = useState<Wish[]>(INITIAL_WISHES);

// Strongly typed DOM reference
const frameRef = useRef<HTMLDivElement>(null);
const audioRef = useRef<HTMLAudioElement | null>(null);

// Form Event Typing
const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  // ...
};
```

4.4 Type Narrowing & Safe Guards

```
// src/utils/assets.ts - Guard clauses guaranteeing string safety
export function asset(path: string): string {
  if (!path) return '';
  if (path.startsWith('http://') || path.startsWith('https://') || path.startsWith('data:')) {
    return path;
  }
  const base = import.meta.env.BASE_URL || '/';
  const cleanBase = base.endsWith('/') ? base : `${base}/`;
  const cleanPath = path.startsWith('/') ? path.slice(1) : path;
  return `${cleanBase}${cleanPath}`;
}
```

5. Framer Motion & Complex Animation Mechanics

Framer Motion powers the fluid, 60fps micro-interactions throughout the site. Rather than using generic CSS transitions, animations are orchestrated declaratively with custom physics and viewport tracking.

5.1 Dynamic Variants with Function Parameters (3D Countdown Flip)

In `CountDown.tsx`, variants are defined as functions that take an index argument, enabling staggered 3D card flips.

```
// src/components/CountDown.tsx
import { motion, type Variants } from 'framer-motion';

const cardVariants: Variants = {
  hidden: { rotateY: 0 },
  visible: (idx: number) => ({
    rotateY: 180,
    transition: {
      duration: 0.85,
      ease: [0.34, 1.4, 0.64, 1], // Custom overshoot spring curve
      delay: 0.75 + idx * 0.14, // Staggered sequential delay
    },
  }),
};

// Applied in JSX:
<motion.div
  custom={idx}
  variants={cardVariants}
  initial="hidden"
  whileInView="visible"
  viewport={{ once: true }}
>
```

5.2 Cubic-Bézier Elastic Curves

To emulate physical inertia, easing curves are defined using cubic-bézier control points:

- `ease: [0.34, 1.56, 0.64, 1]`: Generates an elastic rubber-band overshoot on entrance, where the element moves past its destination and snaps back.
- `ease: [0.25, 1, 0.5, 1]`: Decelerating ease-out curve for natural text reveals.

5.3 Infinite Keyframe Loops (Twinkling Stars & Breathing Florals)

Keyframes animate continuous ambient motion without blocking the main browser thread.

```
// Twinkling Star Keyframe Animation
<motion.div
  className="nce-location-star-one"
  animate={{
    opacity: [0.35, 0.9, 0.35],
    scale: [0.9, 1.05, 0.9]
  }}
  transition={{
    duration: 3.8,
    ease: 'easeInOut',
    repeat: Infinity
  }}
>
  <img alt="Star" src={asset('/svg/star.svg')} />
</motion.div>
```

5.4 Viewport-Driven Scroll Triggering with `useInView`

```
// src/components/Location.tsx
import { useRef } from 'react';
import { motion, useInView } from 'framer-motion';

export default function Location() {
  const frameRef = useRef<HTMLDivElement>(null);
  // Fires animation when 35% of the frame is visible, exactly once
  const isInView = useInView(frameRef, { amount: 0.35, once: true });

  return (
    <div ref={frameRef}>
      <motion.h2
        initial={{ opacity: 0, y: 25 }}
        animate={isInView ? { opacity: 1, y: 0 } : { opacity: 0, y: 25 }}
        transition={{ duration: 1.0, delay: 1.4 }}
      >
        Location
      </motion.h2>
    </div>
  );
}
```

6. CSS Architecture, 3D Transforms & Tailwind

The styling architecture combines utility-first Tailwind classes for layout scaffolding with advanced pure CSS for 3D coordinate transformations.

6.1 3D CSS Flip Card Mechanics

The 3D cards in the Countdown timer and Location ribbon rely on three fundamental CSS 3D properties:

```
/* 1. Establish 3D perspective viewport */
.nce-card-container {
  perspective: 1000px;
}

/* 2. Preserve 3D space across parent and child */
.nce-card-inner {
  position: relative;
  width: 100%;
  height: 100%;
  transform-style: preserve-3d;
  transition: transform 0.85s cubic-bezier(0.34, 1.4, 0.64, 1);
}

/* 3. Hide reverse face when flipped */
.nce-card-front,
.nce-card-back {
  position: absolute;
  inset: 0;
  backface-visibility: hidden;
  -webkit-backface-visibility: hidden;
}

/* 4. Pre-rotate back face */
.nce-card-back {
  transform: rotateY(180deg);
}
```

How perspective Works

`perspective: 1000px` determines the simulated distance between the viewer's eye and the $z=0$ plane. Lower values exaggerate 3D distortion; 1000px delivers a subtle, realistic physical depth.

Role of backface-visibility

Without `backface-visibility: hidden`, when a card flips 180 degrees, the front face would show mirror-reversed content instead of revealing the back face content.

6.2 Glassmorphism & Translucent Backdrop Filter

The navigation header floats over the viewport with frosted glass translucency:

```

/* src/styles/main.css - Line 2754 */
.shared-navbar.fixed.nce-navbar {
  background-color: rgba(244, 229, 227, 0.72);
  backdrop-filter: blur(8px);
  -webkit-backdrop-filter: blur(8px);
  border-bottom: 1px solid rgba(235, 212, 210, 0.45);
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  z-index: 1000;
}

```

6.3 Tailwind Design System Configuration

```

// tailwind.config.js
module.exports = {
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
  theme: {
    extend: {
      colors: {
        sage: '#8a9a86',
        blush: '#f4e5e3',
        rose: '#c88d83',
        ribbon: '#d97d74',
        tan: '#e8d5cf',
      },
      fontFamily: {
        script: ['Galada', 'cursive'],
        serif: ['Playfair Display', 'serif'],
        sans: ['Mulish', 'sans-serif'],
      },
    },
  },
  plugins: [],
};

```

7. Vite Build Engineering, Routing & CI/CD Pipeline

7.1 Subpath Asset Resolution (`import.meta.env.BASE_URL`)

Deploying to GitHub Pages means the application lives at a repository subpath:

`https://ganeshpoojary727.github.io/Code_Nimbus_Solutions_Assignment/`.

Hardcoded root paths like `/svg/star.svg` fail because the browser requests them from the domain root. The project uses a centralized `asset(path)` helper:

```
// src/utils/assets.ts
export function asset(path: string): string {
  if (!path) return '';
  if (path.startsWith('http://') || path.startsWith('https://') || path.startsWith('data:')) {
    return path;
  }
  const base = import.meta.env.BASE_URL || '/';
  const cleanBase = base.endsWith('/') ? base : `${base}/`;
  const cleanPath = path.startsWith('/') ? path.slice(1) : path;
  return `${cleanBase}${cleanPath}`;
}
```

7.2 Why `HashRouter` Instead of `BrowserRouter`?

Architectural Interview Question

"Why did you choose `HashRouter` over `BrowserRouter`?"

GitHub Pages is a static web server without server-side rewrite rules (such as Nginx `try_files $uri /index.html`). If a user bookmarks or refreshes `https://.../gallery` with `BrowserRouter`, GitHub Pages looks for a physical directory named `gallery/index.html` and returns an HTTP 404 error.

`HashRouter` stores route state in the URL hash fragment (`#/gallery`). Browsers never send the hash portion to the web server, allowing React Router to intercept and render the route entirely client-side without requiring server reconfiguration.

7.3 GitHub Actions CI/CD Pipeline (`deploy.yml`)

On every push to the `main` branch, GitHub Actions executes an automated multi-stage build:

```

# .github/workflows/deploy.yml
name: Deploy to GitHub Pages

on:
  push:
    branches: [main]

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Node.js 20
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
          cache-dependency-path: naming-ceremony/package-lock.json

      - name: Install Dependencies & Build
        run: |
          cd naming-ceremony
          npm ci
          npm run build

      - name: Upload GitHub Pages Artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: naming-ceremony/dist

      - name: Deploy to GitHub Pages
        uses: actions/deploy-pages@v4

```

8. Component Walkthrough & Architecture

A complete file-by-file breakdown of every module comprising the application.

File Path	Role	Key Mechanics & Implementation
<code>src/main.tsx</code>	Application Entrypoint	Mounts React 19 root into <code>#root</code> DOM node wrapped with <code><HashRouter></code> .
<code>src/App.tsx</code>	Global Root Shell	Houses <code><Suspense></code> boundary, sticky <code>Navbar</code> , persistent <code>Music</code> controller, and page routes.
<code>src/pages/HomePage.tsx</code>	Page Coordinator	Manages <code>wishes</code> state, imports 9 lazy-loaded sections, handles submission callbacks.
<code>src/components/HeroSection.tsx</code>	Hero Banner	Floral entrance blooms, title typography, baby announcement, and infinite star twinkling loops.
<code>src/components/CountDown.tsx</code>	Countdown Timer	Consumes <code>useCountdown</code> , renders 4 3D flipping cards (Days, Hours, Minutes, Seconds) with staggered delays.
<code>src/hooks/useCountdown.ts</code>	Custom Timer Hook	Pure mathematical diff calculation, interval ticker, <code>padStart(2, '0')</code> formatting, memory leak cleanup.
<code>src/components/WarmInvite.tsx</code>	Invitation Letter	Personal message from parents, decorative floral SVGs, responsive layout framing.
<code>src/components/Location.tsx</code>	Venue & Maps	3D ribbon flip reveal, venue address, venue photograph, and interactive Google Maps link.
<code>src/components/Wishes.tsx</code>	Community Carousel	Horizontally scrollable/paginated list of warm wishes from friends and relatives.
<code>src/components/SendWishes.tsx</code>	Interactive Form	Controlled form inputs, AI random wish generator, submission loading state, and state lifting to parent.
<code>src/components/Schedule.tsx</code>	Event Timeline	Chronological ceremony schedule with custom touch/mouse drag-swipe gesture detection via <code>useRef</code> .
<code>src/components/Video.tsx</code>	Video Showcase	Responsive 16:9 embedded YouTube video container celebrating previous milestone moments.
<code>src/components/Music.tsx</code>	Audio Controller	Floating bottom-corner music button, HTML5 <code><audio></code> ref management, browser autoplay handling.
<code>src/components/Navbar.tsx</code>	Navigation Header	Fixed glassmorphism bar with active route highlighting using React Router's <code>NavLink</code> .
<code>src/components/Footer.tsx</code>	Site Footer	Traditional closing blessings, copyright notice, and links to top.
<code>src/data/content.ts</code>	Data Store	Centralized configuration for event dates, venues, parents' names, initial wishes, and AI templates.
<code>src/utils/assets.ts</code>	Asset URL Resolver	Prepends <code>import.meta.env.BASE_URL</code> to ensure assets load correctly across local dev and GitHub Pages.

9. Technical Interview Defense Cheatsheet

Common senior frontend interview questions and the exact architectural defense based on this codebase:

Q1: How did you ensure 60fps performance with so many concurrent animations?

All Framer Motion animations animate hardware-accelerated CSS properties: `transform` (such as `scale`, `rotateY`, `y`) and `opacity`. These properties do not trigger expensive browser layout recalculations or repaints. They are offloaded directly to the GPU compositor layer. Furthermore, lazy loading ensures heavy asset trees aren't competing for initial render cycles.

Q2: Why use a custom hook (`useCountdown`) instead of putting the timer inside the component?

Separation of concerns. The mathematical countdown calculation and interval lifecycle are pure business logic. Extracting them into `useCountdown` makes the logic unit-testable in isolation, highly reusable across different views, and prevents the `CountDown.tsx` UI component from being cluttered with interval timers and state math.

Q3: How does the AI Wish Generator work in `SendWishes.tsx`?

Inside `src/data/content.ts`, a curated set of heartfelt blessing templates (`AI_WISH_TEMPLATES`) is maintained. When the user clicks the "Generate AI Wish" button, `handleGenerateAIWish` selects a pseudo-random template using `Math.floor(Math.random() * AI_WISH_TEMPLATES.length)` and updates the controlled message state, instantly populating the textarea.

Q4: How did you avoid memory leaks with `setInterval`?

In React, unmounting a component while an interval is running leaves an orphaned timer in browser memory attempting to update unmounted component state. In `useCountdown.ts`, the `useEffect` returns a cleanup closure: `return () => clearInterval(interval);`. React automatically executes this cleanup function whenever the component unmounts or before re-running the effect.

Q5: How does swipe/drag work in `Schedule.tsx` without external gesture libraries?

I implemented a lightweight pointer tracking system using React synthetic touch and mouse events (`onTouchStart`, `onTouchEnd`, `onMouseDown`, `onMouseUp`). A `useRef<number | null>(null)` stores the initial client X coordinate without triggering component re-renders. On touch/mouse release, the delta is measured against a 40px threshold to trigger forward or backward timeline progression.

Q6: What is the purpose of `Oxlint` in this project?

`Oxlint` is an ultra-fast, Rust-based JavaScript and TypeScript linter that runs 50 to 100 times faster than legacy `ESLint`. It caught dead imports and unused variables instantly during development, ensuring that our final codebase had zero warnings and zero dead dependencies before submission.

Q7: How did you handle browser autoplay audio restrictions in `Music.tsx`?

Modern browsers prohibit programmatic audio playback without prior user interaction. In `Music.tsx`, instead of attempting forbidden autoplay on mount, an explicit user-interactive floating toggle button was created. The playback call catches any browser rejection gracefully: `audioRef.current.play().catch(err => console.warn(err))`.

Naming Ceremony Web Application - Master Technical Documentation

Prepared by Ganesh Poojary • Candidate for Front-End Developer Intern • Code Nimbus Solutions Assignment